Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Vize sonrası değerlendirme: kopya ve derse katılım üzerine uyarılar.
- Graf (graph) veri yapısı: neden doğrusal olmayan bir yapıdır, gerçek hayattaki kullanım alanları.
- Graf terminolojisi: düğüm, kenar, komşuluk, derece, yol, döngü, bağlı bileşen, ağaç, yayılan ağaç, klik, bağlı/bağlı olmayan graf, iki parçalı graf, ağırlıklı graf.
- Sembol tablosu ve sembol graf.
- Grafın saklanması: komşuluk matrisi (adjacency matrix) ve komşuluk listesi (adjacency list); yoğun ve seyrek graflar; C ile komşuluk listesi.
- Yönlü graf (directed graph, digraph): giriş derecesi ve çıkış derecesi.
- Minimum yayılan ağaç (minimum spanning tree, MST): Prim ve Kruskal algoritmaları, döngü kontrolü için etiket dizisi, karmaşıklıklar.

Hocanın Özellikle Vurguladığı Kısımlar

- Hoca vizede çok fazla kopya çekildiğini söyledi. Bazı cevaplar internetten kopyalanıp yapıştırılmış kitap cümleleriydi.
  - COVID döneminde de aynısı olmuş. Bir yıl sonra yüz yüze derse dönen öğrencilerin bilgisi çok yetersiz kalmış.
  - İş hayatında yanınızda kopya çekeceğiniz kimse olmayacak. Ödev ve sınavları kendi başınıza yapın.
- Diğer öğretim üyeleri de derse katılımın çok düşük olduğunu söylüyor. Bir öğrenci, kayıtların sonradan hızlandırılarak izlenmesini sebep olarak gösterdi.
- Dizi, bağlı liste, yığın ve kuyruk doğrusal (linear) yapılardır. Graf doğrusal değildir: bilgiler aralarındaki ilişkiye göre birbirine bağlanır, sıralı bir erişim yoktur.
- Graf, günlük hayatta en çok kullanılan veri yapılarından biridir. En temel örnek internettir.
- Düğümlerin ve kenarların neyi göstereceğine uygulamaya göre siz karar verirsiniz.
- Kod yazarken düğümler isimlerle değil sayılarla (0, 1, 2, …) temsil edilmelidir. Kullanıcıya gösterirken sembol tablosuyla isme çevrilir.
- Seyrek bir grafı matriste saklamak bellek açısından kötü bir seçimdir. Seyrek graflar için komşuluk listesi kullanılmalıdır.
- Graf gezinme (traversal), en kısa yol (Dijkstra) gibi konular uzaktan eğitim nedeniyle bu dönem büyük ihtimalle anlatılamayacak, gelecek yıl görülecek.

Kısa Tekrar Notları

- **Düğüm (vertex, node):** Graftaki bir varlık (kişi, router, kavşak, sayfa vb.).
- **Kenar (edge):** İki düğüm arasındaki bağlantı. u ve v'yi bağlayan kenar (u, v) diye yazılır.
- **Komşu (adjacent):** Aralarında kenar olan iki düğüm.
- **Derece (degree):** Bir düğüme bağlı kenar sayısı.
- **Yol (path):** Bir düğümden başka bir düğüme kenarlar üzerinden giden dizi. Uzunluğu kenar sayısıdır.
- **Döngü (cycle):** Başlanan düğüme geri dönen yol.
- **Bağlı bileşen (connected component):** Birbirine bağlı düğümlerin oluşturduğu parça (ör. kıtalar).

- **Bağlı graf (connected graph):** Tüm düğümlerin aynı bileşende olduğu graf. Değilse bağlı olmayan (disconnected) graftır.
- **Ağaç (tree):** Bağlı ve döngüsüz (acyclic) graf.
- **Yayılan ağaç (spanning tree):** Graftaki tüm düğümleri içeren, döngüsüz alt graf.
- **Klik (clique):** Tüm düğümlerin birbirine bağlı olduğu alt graf (ya da grafın kendisi).
- **İki parçalı graf (bipartite graph):** Düğümler iki kümeye ayrılır. Kenarlar sadece farklı kümeler arasındadır (ör. öğrenciler ve dersler, üyeler ve filmler).
- **Ağırlıklı graf (edge-weighted graph):** Kenarların sayısal değeri vardır (mesafe, süre vb.).
- **Yönlü graf (digraph):** Kenarların yönü vardır. $u \to v$ olması $v \to u$ olduğu anlamına gelmez.
  - **Giriş derecesi (in-degree):** Düğüme gelen kenar sayısı.
  - **Çıkış derecesi (out-degree):** Düğümden çıkan kenar sayısı.
- **Komşuluk matrisi:** $V \times V$ matris. Yönsüz grafta köşegene göre simetriktir. Ağırlıksız grafta 1/0, ağırlıklı grafta kenar ağırlığı tutulur.
- **Komşuluk listesi:** Her düğüm için komşularının bağlı listesi.
- **Yer ve işlem karmaşıklıkları:**

İşlem	Komşuluk matrisi	Komşuluk listesi
Bellek	$O(V^2)$	$O(V+E)$
u ile v arasında kenar var mı?	$O(1)$	$O(\deg(u))$
v'nin tüm komşularını listele	$O(V)$	$O(\deg(v))$

- **MST:** Ağırlıklı bir grafta kenar ağırlıkları toplamı en küçük olan yayılan ağaç. n düğümlü bir grafın MST'sinde $n-1$ kenar vardır.
- **Prim:** Bir düğümden başlar. Her adımda ağaca bağlı düğümlerden çıkan, döngü oluşturmayan en ucuz kenarı ekler.
- **Kruskal:** Tüm kenarları küçükten büyüğe sıralar. Sırayla, iki farklı bağlı bileşeni birleştiren (döngü oluşturmayan) kenarları ekler.

Detaylı Açıklamalar

Graf nerelerde kullanılır?

- **İnternet:** Düğümler router'lar, kenarlar aralarındaki fiziksel ya da sanal bağlantılar. Hoca, internetin 1997-2021 arasındaki gelişimini gösteren bir görselleştirme videosu (Opte Project) izletti.
- **Haritalar (ör. Google Maps):** Düğümler kavşaklar, kenarlar yollar. Kenarlarda mesafe, süre, trafik durumu, taşıt türü bilgisi tutulabilir. "Kadıköy'den Beşiktaş'a en kısa/en hızlı nasıl giderim?" soruları graf üzerinde çözülür. Vapurla ya da karayoluyla gitmek gibi birden fazla alternatif olduğu için yapı doğrusal değildir.
- **Sosyal ağlar:** Düğümler kişiler, kenarlar arkadaşlık, iş arkadaşlığı, akrabalık gibi ilişkiler.
- **İletişim:** Telefonlar/bilgisayarlar ve aralarındaki kablolar.
- **Devreler:** Kapılar, register'lar, işlemciler ve aralarındaki teller.
- **Web:** Sayfalar ve sayfalar arası köprüler (hyperlink). Bu yönlü bir graftır.
- **Salgın takibi:** Kişiler ve hastalığın kimden kime geçtiği. Bu da yönlü bir graftır.

Terminoloji örnekleri

- Bir örnek grafta A'nın sadece D'ye bağlantısı var, derecesi 1. D; A, B, C ve E'ye bağlı, derecesi 4.
- Yönsüz graf: Arkadaşlık gibi ilişkiler. Elif Ayşe'nin arkadaşıysa Ayşe de Elif'in arkadaşıdır.
- Yönlü graf: Karayolu haritası. Kadıköy'den Beşiktaş'a giderken kullanılan yollar dönüşte aynı olmayabilir (tek yönlü yollar, şeritler).
- Bir ağaç aynı zamanda bir graftır. Ama döngü içeren bir graf ağaç değildir.
- Bağlı olmayan graf örneği: Sadece fakülteleri gösteren bir üniversite grafı. Her fakülte kendi içinde bağlı, fakülteler arasında bağlantı yok.

Sembol tablosu ve sembol graf

- Algoritmalar düğümleri sayılarla (0, 1, 2, ...) işler. Gerçek isimler bir string dizisinde tutulur.
  - Ör. 0 → Ali, 1 → Ayşe, 2 → Banu.
- Ters yönde erişim için (isimden sayıya) ayrı bir indeks de tutulabilir.
- Örnek sembol graf: Havaalanları. JFK 0, Houston 4 gibi numaralanır. Hesaplar numaralarla yapılır, kullanıcıya isimler gösterilir.

Komşuluk matrisi

- Ağırlıksız grafta `A[i][j] = 1` ise i ile j arasında kenar vardır, 0 ise yoktur.
- Ağırlıklı grafta `A[i][j]` kenarın ağırlığını tutar (ör. 0-1 arası 2, 0-2 arası 3).
- Yönsüz grafta matris köşegene göre simetriktir. Ör. 0. düğüm 1, 2, 5, 6'ya bağlıysa hem 0. satırda hem 0. sütunda bu gözler doludur.
- Küçük örnek (düğümler 1, 2, 3; kenarlar 1-2 ve 1-3):

	1	2	3
1	0	1	1
2	1	0	0
3	1	0	0

Yoğun ve seyrek graf

- **Yoğun graf (dense):** Düğümler arasında çok sayıda bağlantı var (ör. bir şehrin yol ağı).
- **Seyrek graf (sparse):** Bağlantılar az.
  - Facebook'ta 1 milyar kullanıcı var ama bir kişinin en fazla birkaç yüz arkadaşı var. Matriste her satırın 1 milyar gözünden sadece birkaç yüzü dolu olur.
  - Netflix örneği: 1 milyar üye ve 10.000 film/dizi. Bir üye en fazla birkaç yüz tanesini izler. Bu aynı zamanda iki parçalı bir graftır.
- Seyrek grafı matriste saklamak çok yer israf eder. Komşuluk listesi kullanılmalıdır.

Komşuluk listesi

- Her düğüm için bir bağlı liste tutulur. Listede o düğümün komşuları vardır.
- Her düğümün komşu sayısı farklı olduğu için değişken uzunluklu bir yapı gerekir. Bu yüzden bağlı liste uygundur.
- Örnek: 0 → 5, 2, 1, 6; 1 → 0; 3 → 5, 4; 5 → 0, 4, 3.
- Yeni komşu listenin başına eklenir. Böylece sona kadar gitmek gerekmez.

```c
struct node {
    int vertex;
    struct node *next;
};

struct Graph {
    int numVertices;
    struct node **adjLists;   /* her düğüm için liste başı pointer'ı */
};

struct node *createNode(int v) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

struct Graph *createGraph(int vertices) {
    struct Graph *graph = malloc(sizeof(struct Graph));
```

```
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct node *));
    for (int i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;          /* başlangıçta tüm listeler boş */
    return graph;
}

void addEdge(struct Graph *graph, int u, int v) {
    struct node *newNode = createNode(v);   /* u'nun listesine v */
    newNode->next = graph->adjLists[u];
    graph->adjLists[u] = newNode;

    newNode = createNode(u);                /* yönsüz graf: v'nin listesine u */
    newNode->next = graph->adjLists[v];
    graph->adjLists[v] = newNode;
}
```

- `adjLists` bir pointer dizisidir. Elemanları liste başlarının adresleridir, bu yüzden tipi `struct node **`'dır.
- Hocanın izleme örneği: `addEdge(graph, 0, 2)` çağrılıyor.
  - 2 için yeni düğüm (adres 2500) oluşturulur. `next`'i 0'ın listesinin eski başıdır (NULL). `adjLists[0]` artık 2500'ü gösterir.
  - 0 için yeni düğüm (adres 3000) oluşturulur. `next`'i 2'nin listesinin eski başıdır (NULL). `adjLists[2]` artık 3000'i gösterir.
  - Sonra eklenen her yeni komşu ilgili listenin başına gelir, eski baş onun `next`'i olur.

Matris ile listenin karşılaştırılması

- **Bellek:** Matris V^2 yer tutar. Liste $V + E$ yer tutar (yönsüz grafta her kenar iki listede görünür). Hiçbir yere bağlı olmayan bir düğüm için bile liste başı tutulur.
- **"u ile v arasında kenar var mı?"** Matriste doğrudan `A[u][v]`'ye bakılır, $O(1)$. Listede u'nun listesi baştan sona taranabilir.
- **"v kimlere doğrudan bağlı?"** Matriste v'nin satırındaki tüm V göz taranır. Listede sadece v'nin listesindeki elemanlar okunur.
- Öğrenci sorusu: 0'dan 2'ye dolaylı bir yol olup olmadığı nasıl bulunur? Hoca bunun ayrı bir konu olduğunu, graf gezinme algoritmalarıyla çözüldüğünü söyledi. Buradaki soru sadece doğrudan komşulardır.

Yönlü graf (digraph)

- Kenarların yönü vardır. Komşuluk matrisi artık simetrik olmak zorunda değildir: `A[u][v]` ile `A[v][u]` farklı olabilir.
- Komşuluk listesinde sadece çıkan kenarlar tutulur. Ör. 0 → 1, 2, 5, 6; ama 5 → 3, 4 (5 artık 0'a bağlı görünmez).
- Örnekte 6. düğümün giriş derecesi 1, çıkış derecesi 3'tür.

Minimum yayılan ağaç (MST)

- Örnek kullanım: Bir ağdaki tüm bilgisayarları birbirine bağlamak ama bunu en kısa toplam kablo uzunluğuyla yapmak. Herkes herkese bağlı olmak zorunda değildir, ama herkes ağa bağlı olmalıdır.
- Örnek graf ve kenar ağırlıkları:

Kenar	Ağırlık	Kenar	Ağırlık
3-5	18	3-4	34
1-7	21	4-5	40
6-7	25	4-7	46
0-2	29	0-6	51
0-7	31	4-6	51

Kenar	Ağırlık	Kenar	Ağırlık
0-1	32	0-5	60

Prim algoritması (1930)

- Açgözlü (greedy) bir algoritmadır: her adımda o an en iyi görünen seçimi yapar.
- 0'dan başlanır. Her adımda ağaçtaki düğümlerden çıkan ve döngü oluşturmayan en ucuz kenar eklenir.

Adım	Eklenen kenar	Ağırlık	Not
1	0-2	29	0'ın kenarları: 29, 31, 32, 51, 60
2	0-7	31	2'nin başka kenarı yok
3	7-1	21	7'nin kenarları aday oldu. 0-1 artık döngü oluşturur
4	7-6	25	0-6 artık döngü oluşturur
5	7-4	46	4-6 (51) artık döngü oluşturur
6	4-3	34	4'ün kenarları: 34, 40
7	3-5	18	Tüm düğümler bağlandı

- Toplam ağırlık: $29 + 31 + 21 + 25 + 46 + 34 + 18 = 204$.

Kruskal algoritması

- Tüm kenarlar küçükten büyüğe sıralanır. Sırayla bakılır. Kenar döngü oluşturmuyorsa eklenir.
- Prim'den farkı: Bir düğümden genişlemez, tüm kenarlara global olarak bakar.

Kenar	Ağırlık	Karar
3-5	18	al
1-7	21	al
6-7	25	al
0-2	29	al
0-7	31	al
0-1	32	alma (0-7-1 ile döngü)
3-4	34	al
4-5	40	alma (4-3-5 ile döngü)
4-7	46	al, 7 kenar oldu, bitti

- Sonuç Prim ile aynı ağaçtır (toplam 204).

Döngü kontrolü: etiket dizisi

- Bir kenar eklenirken iki uç düğümün aynı bağlı bileşende olup olmadığı kontrol edilir. Aynı bileşendeyse kenar döngü oluşturur.
- Her düğüm için bir etiket (label) tutulur. Başlangıçta tüm etiketler 0'dır (hiçbir bileşende değil).
- (u, v) kenarı değerlendirilirken:
  - **İkisi de 0:** Yeni bir etiket verilir, kenar alınır.
  - **Biri 0, diğeri x:** 0 olana x verilir, kenar alınır.
  - **Biri x, diğeri y ($x \neq y$):** Farklı bileşenlerdir. Tüm y etiketleri x yapılır (ya da tersi), kenar alınır.
  - **İkisi de x:** Aynı bileşendedir. Kenar alınmaz.
- Örnek: 0-1, 1-2 ve 2-3 alınmışsa 0 ile 3 aynı bileşendedir, 0-3 alınmaz. Ama {0, 1} ve {2, 3} iki ayrı bileşense 1-2 alınabilir, sadece iki bileşeni birleştirir.

Karmaşıklıklar

- **Prim (komşuluk matrisi ile):** $V - 1$ adım. Her adımda en ucuz kenarı bulmak ve mesafeleri güncellemek $O(V)$. Toplam $O(V^2)$.
- **Kruskal:** Kenarları sıralamak $O(E \log E) = O(E \log V)$.

Ders sonrası notlar

- **Prim her zaman doğru sonuç verir.** Hoca "Prim başlangıç noktasına bağlı olarak minimum olmayan bir yayılan ağaç bulabilir" dedi. Bu yanlıştır. Bağlı, ağırlıklı, yönsüz bir grafta Prim hangi düğümden başlarsa başlasın bir MST bulur. Kruskal da her zaman bir MST bulur. Aynı ağırlıkta kenarlar varsa farklı başlangıçlar farklı ama eşit toplamlı MST'ler verebilir.
- **Açgözlü yaklaşım her problemde en iyi sonucu vermez.** Hoca açgözlü yaklaşımı "lokalde en iyi olan globalde de en iyidir" diye tanımladı. Bu genel bir kural değildir. MST için doğruluğu ispatlanmıştır (kesme özelliği, cut property). Başka problemlerde açgözlü seçim en iyi sonucu vermeyebilir.
- **Kruskal ilk anlatımda yanlış ilerledi.** Hoca 0-1 kenarını da aldı. Bir öğrenci bunun 0-7-1 ile döngü oluşturduğunu fark etti, hoca düzeltti.
- **Komşuluk listesinde komşuları listeleme $O(1)$ değildir.** Hoca "v kimlere bağlı" sorusunun listede $O(1)$ olduğunu söyledi. Doğrusu $O(\deg(v))$'dir: listede kaç komşu varsa o kadar eleman okunur. Matristeki $O(V)$'den genellikle çok daha azdır.
- **Kenar sorgusu:** Listede "u-v kenarı var mı" sorusu en kötü durumda u'nun derecesi kadar sürer, $O(\deg(u))$. Bu en fazla $V - 1$'dir.
- **Hangisi daha hızlı?** Hoca Kruskal'ın daha hızlı olduğunu söyledi. Bu grafa bağlıdır. Yoğun graflarda ($E \approx V^2$) matrisli Prim'in $O(V^2)$'si, Kruskal'ın $O(E \log V) = O(V^2 \log V)$'sinden iyidir. Seyrek graflarda Kruskal ya da heap'li Prim ($O(E \log V)$) daha iyidir.
- **Etiket dizisinin maliyeti:** İki bileşen birleşirken tüm etiketleri değiştirmek $O(V)$ sürer. Pratikte bunun yerine union-find (disjoint set) yapısı kullanılır, birleştirme ve kontrol neredeyse sabit zamanlıdır.
- **Ağaç tanımı:** Hoca bir yerde "döngü yoksa ağaçtır" dedi. Tam tanım bağlı ve döngüsüz olmaktır. Döngüsüz ama bağlı olmayan graf ormandır (forest).
- **Altyazı:** 00:11 civarında izletilen İngilizce video ve aradaki bozuk satırlar ("Altyazı M." vb.) ders içeriği değildir.

Duyurular

- Ertesi gün yeni bir ödev verilecek.
- Kruskal algoritmasının kodu bir sonraki derste yazılacak.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.